SE446: Big Data Engineering

Week 9B: Spark Structured Streaming — Real-Time Data Processing

Prof. Anis Koubaa

SE 446
Alfaisal University
https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

1. The Streaming Mental Model
2. Reading Streams
3. Transformations on Streams
4. Writing Stream Results
5. End-to-End Pipeline
6. Summary

Outline

1. The Streaming Mental Model
2. Reading Streams
3. Transformations on Streams
4. Writing Stream Results
5. End-to-End Pipeline
6. Summary

Learning Objectives

By the end of this session you will be able to:

1. Explain the "infinite table" model of Structured Streaming
2. Distinguish between micro-batch and continuous processing
3. Read from Kafka in Spark using `readStream`
4. Apply DataFrame transformations on streaming data
5. Write results with `writeStream` using append, update, and complete modes
6. Implement tumbling and sliding time windows

The Big Idea

You already know the DataFrame API from Weeks 7–8. **Structured Streaming uses the exact same API** — the only difference is the input keeps growing.

The Big Data Journey — Where Are We?

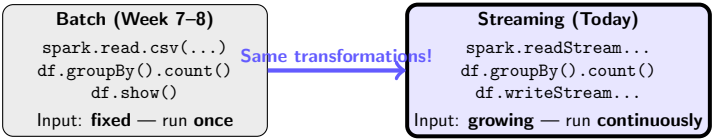
Session 9A Recap

Kafka = distributed message bus.
Producers → Topics/Partitions → Consumers.

Session 9B Goal

Connect Spark to Kafka and process streams in **real time**.

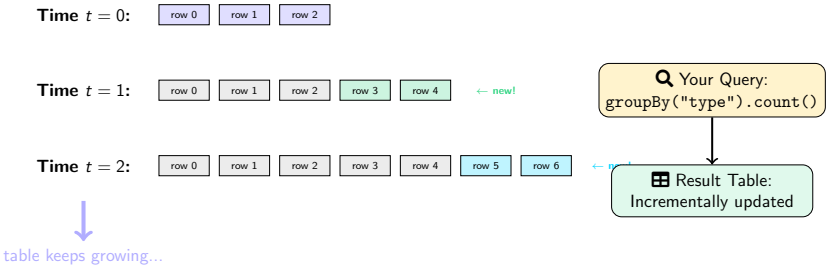
From Batch to Streaming — Same API!



The Key Insight

You do **not** need to learn a new API for streaming. `filter()`, `groupBy()`, `join()` — everything works the same. Only the input source and output sink change.

The Infinite Table Analogy

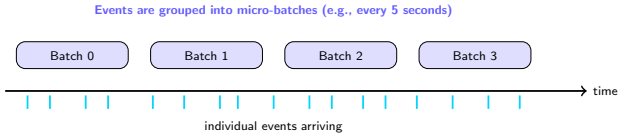


How It Works

Spark processes each **micro-batch** of new rows, then **incrementally updates** the result. It does **not** recompute from scratch every time.

Micro-Batch vs Continuous Processing

Mode	How It Works	Latency
Micro-batch (default)	Accumulates data, processes as a small Spark batch	100ms – few seconds
Continuous (experimental)	Event-by-event processing, limited operations	∼1ms



For This Course

We use **micro-batch** mode (the default). It is stable, well-tested, and handles most real-world use cases. For sub-millisecond latency, look at Apache Flink.

Outline

1 The Streaming Mental Model

2 Reading Streams

3 Transformations on Streams

4 Writing Stream Results

5 End-to-End Pipeline

6 Summary

readStream — The Streaming Entry Point

```
# BATCH read (familiar from Weeks 7-8):
df = spark.read.csv("hdfs:///data/crimes.csv", header=True)

# STREAMING read from Kafka (new!):
stream_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "crimes") \
    .option("startingOffsets", "latest") \
    .load()
```

spark.read (Batch)

Returns a **static** DataFrame
(finite, all data exists now)

spark.readStream (Streaming)

Returns a **streaming** DataFrame
(infinite, new rows keep arriving)

Kafka Message Schema

The raw DataFrame has columns: `key`, `value` (both bytes), `topic`, `partition`, `offset`, `timestamp`.
You must **parse** the `value` column yourself.

Parsing Kafka Messages

Kafka stores everything as bytes. You must define a schema and parse:

```
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StringType, IntegerType, BooleanType

# Define the expected JSON schema
crime_schema = StructType() \
    .add("id", IntegerType()) \
    .add("type", StringType()) \
    .add("district", IntegerType()) \
    .add("hour", IntegerType()) \
    .add("arrest", BooleanType())

# Parse: bytes -> string -> JSON -> DataFrame columns
crimes_df = stream_df \
    .selectExpr("CAST(value AS STRING) as json_str") \
    .select(from_json(col("json_str"), crime_schema).alias("data")) \
    .select("data.*")

# Now crimes_df has columns: id, type, district, hour, arrest
```

Three-Step Pattern

(1) Cast bytes to string → **(2)** Parse JSON with schema → **(3)** Flatten nested struct into columns.

Streaming Sources Comparison

Source	Format	Use Case	Fault-Tolerant?
`kafka`	Messages from topics	**Production streaming**	✔ Yes
`socket`	Text from TCP	Testing only	✖ No
`rate`	Generated rows/sec	Benchmarking	✔ Yes
`file`	New files in dir	Semi-batch (watch folder)	✔ Yes

For This Course

We use **Kafka** as our streaming source. This is the standard production pattern: `Kafka` → `Spark Structured Streaming` → output.

Recall from Session 9A

Our Kafka broker runs at `134.209.172.50:9092`. We created topics and Python

Outline

1 The Streaming Mental Model

2 Reading Streams

3 Transformations on Streams

4 Writing Stream Results

5 End-to-End Pipeline

6 Summary

Familiar Operations on Streaming Data

```
# Filter: only thefts (same as batch!)
thefts = crimes_df.filter(col("type") == "THEFT")

# Aggregation: count by crime type
type_counts = crimes_df.groupBy("type").count()

# Aggregation: count by district
district_counts = crimes_df.groupBy("district").count()

# Multiple aggregations
stats = crimes_df.groupBy("type").agg(
    count("*").alias("total"),
    avg("hour").alias("avg_hour")
)
```

The Entire Point of Structured Streaming

This code is **identical** to batch Spark SQL. You already know how to do this from Week 7. The engine handles the "streaming" part automatically.

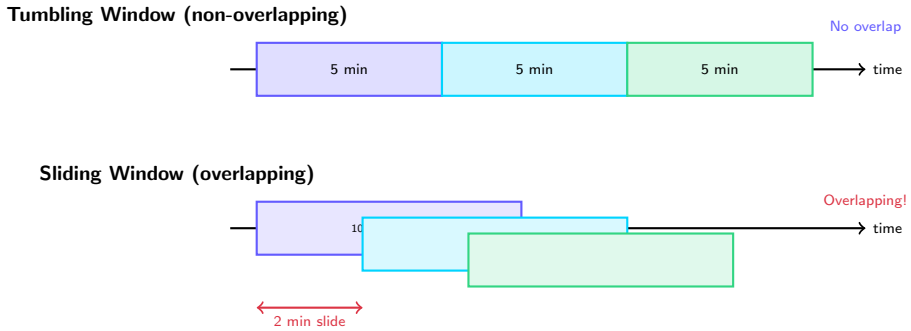
What You CAN and CANNOT Do on Streams

Operation	OK?	Notes
`select`, `filter`, `map`	✓	Stateless — always works
`groupBy().count()`	✓	Stateful — uses complete/update mode
Join (stream + static table)	✓	Static table must fit in memory
Join (stream + stream)	✓	Must define watermark
`orderBy` / `sort`	✗	Cannot sort an infinite table
`distinct`	✗	Would require infinite memory
`limit(n)`	✗	Not defined on infinite data

Why These Limitations?

Think about it: How do you "sort" a table that never stops growing? You would need to wait until all data arrives — but it **never** all arrives. Operations that need to see **all** data are impossible on infinite streams.

Time Windows — Tumbling vs Sliding



Tumbling

Each event belongs to **exactly one** window.
Use for: fixed reporting intervals.

Sliding

Events can belong to **multiple** windows.
Use for: moving averages, trend detection.

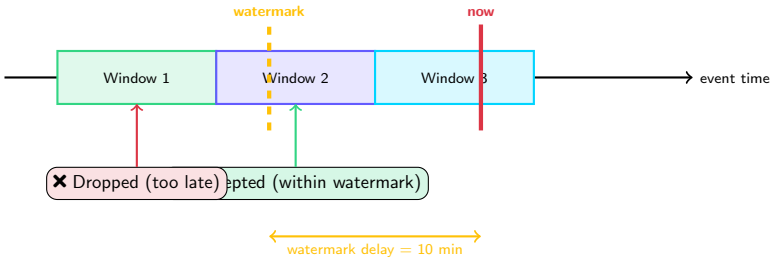
Window Aggregations in Code

```
from pyspark.sql.functions import window, col, count

# TUMBLING window: count crimes per 5-minute window
tumbling = crimes_df \
    .groupBy(
        window("event_time", "5 minutes"),
        "type"
    ).count()

# SLIDING window: 10-min window, sliding every 2 minutes
sliding = crimes_df \
    .groupBy(
        window("event_time", "10 minutes", "2 minutes"),
        "type"
    ).count()
```

Important: You Need a Timestamp Column!

The `window()` function requires an event-time column (e.g., `event_time`). If your JSON has a timestamp, parse it. Otherwise, use the Kafka `timestamp` column.

Watermarks — Handling Late Data



Without Watermark

Spark keeps **all** window state forever. Memory grows **unbounded**!

With Watermark

Spark drops old state once the watermark passes. Memory is **bounded**.

Watermarks in Code

```
# Accept events up to 10 minutes late
# Drop state for windows older than the watermark
crimes_with_watermark = crimes_df \
    .withWatermark("event_time", "10 minutes") \
    .groupBy(
        window("event_time", "5 minutes"),
        "type"
    ).count()
```

When to Use Watermarks

- Always use watermarks with **window aggregations** in production
- The delay value depends on your data: IoT sensors → seconds; mobile apps → minutes; batch uploads → hours
- For this lab: `10 minutes` is a reasonable default

Remember from Week 8

This is similar to `.cache()` controlling memory usage. Watermarks control **how long Spark remembers** partial results.

Outline

1 The Streaming Mental Model

2 Reading Streams

3 Transformations on Streams

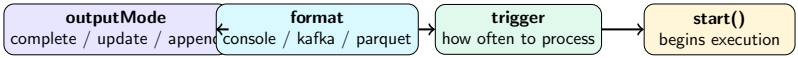
4 Writing Stream Results

5 End-to-End Pipeline

6 Summary

writeStream — The Streaming Output

```
query = type_counts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .trigger(processingTime="5 seconds") \
    .start()

query.awaitTermination()     # block until query is stopped
```



The Streaming Pipeline

`readStream` (source) → **transformations** (filter, groupBy, window) → `writeStream` (sink). This is the complete pattern for every streaming job.

Output Modes Explained

Common Mistake

Using `complete` mode with a high-cardinality `groupBy` (e.g., grouping by user ID). This re-writes **millions of rows** every micro-batch. Use `update` mode instead.

For the Lab

Use `complete` mode for crime type counts (few groups). Use `update` mode for district-level counts (moderate cardinality).

Output Sinks and Trigger Options

Output Sinks

Sink	Best For
`console`	Debugging
`memory`	Interactive SQL queries
`kafka`	Chaining pipelines
`parquet`	Archiving to HDFS
`foreach`	Custom logic (DB, HTTP)

Trigger Options

Trigger	Behavior
`"5 seconds"`	Every 5 seconds
`"0 seconds"`	ASAP (lowest latency)
`once=True`	Run once, then stop
`availableNow`	Process all pending, stop

The Memory Sink — Great for Exploration

Write to an in-memory table, then query it with `spark.sql("SELECT * FROM crimes_live")`. Perfect for notebooks and debugging — **not for production**.

Outline

1 The Streaming Mental Model

2 Reading Streams

3 Transformations on Streams

4 Writing Stream Results

5 End-to-End Pipeline

6 Summary

The Full Pipeline — Putting It All Together

Lab Workflow

1. **Terminal 1**: Run `crime_producer.py` (sends events to Kafka every second)
2. **Terminal 2**: Run Spark streaming job (reads from Kafka, aggregates, prints)
3. **Watch**: Counts update every 5 seconds as new events arrive

Monitoring — Query Progress

```
query = type_counts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .trigger(processingTime="5 seconds") \
    .start()

# Check query status
print(query.status)           # is it running? waiting for data?
print(query.lastProgress)     # timing, rows processed, state info
print(query.recentProgress)   # list of recent micro-batches
```

Spark UI: Streaming Tab

- Input rate (rows/sec)
- Processing rate (rows/sec)
- Batch duration (ms)

Key Metric

If **processing rate** $<$ **input rate**,
your pipeline is falling behind!
Fix: more partitions or faster logic.

Recall from Week 8

The Spark Web UI at `http://master:4040` now has a **Structured Streaming** tab alongside the Jobs/Stages/Storage tabs you already know.

Outline

1 The Streaming Mental Model

2 Reading Streams

3 Transformations on Streams

4 Writing Stream Results

5 End-to-End Pipeline

6 Summary

Key Takeaways

1. **Structured Streaming** = "infinite table" processed with the **same DataFrame API** you already know from Week 7–8
2. `readStream` + `writeStream` replace `read` + `write`. Everything in between is identical.
3. Three output modes: **append** (new rows), **update** (changed), **complete** (full rewrite)
4. **Time windows** (tumbling/sliding) enable real-time aggregation over fixed intervals
5. **Watermarks** bound late data and prevent unbounded memory growth
6. **Kafka** $\rightarrow$ **Spark Streaming** is the standard real-time pipeline

The Week 9 Stack

Kafka (transport) + **Spark Structured Streaming** (processing)

What's Next

Week 10: Midterm 2 + Advanced Topics Review

- Midterm 2 covers Weeks 7–9 (Spark, MLlib, Kafka, Streaming)
- Review all pipeline patterns: batch vs real-time
- Milestone M4 (Spark) due end of Week 10

Today's Lab: Spark Streaming with Kafka

- Write a Python producer that simulates live crime events
- Create a Spark Structured Streaming job reading from Kafka
- Compute real-time crime counts per type (5-min tumbling windows)
- Compute running arrest rate per district
- Observe micro-batch execution in the Spark UI

Milestone M5 Reminder

Due Week 12: End-to-end streaming pipeline (Kafka $\rightarrow$ Spark Streaming $\rightarrow$ output). This lab gives you the building blocks.